

嵌入式智能系统-实验 1

计算机 2303 尤颢辰

March 2026

1 问题陈述

安装 ADS1.2, 在 CodeWarrior 中 make(汇编、连接) 该程序, 并将映像文件装入 AXD 中模拟执行
观察映像文件的组织:

(1) 观察各段的安排次序。

(2) 修改 RO Base 为 0x800, 汇编后再观察各段的次序、地址。

(3) 修改 RW Base 为 0xC00, 汇编后再观察各段的次序、地址。

(4) 上述观察各段地址时, 应观察各段在主存的存放地址, 以及 6 个链接器的符号常量指示的地址, 如果两者不同, 解释原因。

(5) 调整源代码中各段顺序, 汇编后再观察各段的次序、地址。

(6) 改变各段对齐方式, 改变各数据段长度, 再进行观察, 总结对齐规律。

观察伪指令 “LDR R0,=0x53002345” 及其它指令

(1) 有没有文字池, 若有, 观察文字池的地址及内容。

(2) 观察这些指令对应的机器指令, 说明对应的汇编指令及寻址方式

增加其它感兴趣的指令, 比如堆栈操作、转移等指令, 观察其机器指令情况。

单步执行, 观察寄存器情况。

观察各段地址的解释:

将目标代码 (.axf) 装入 AXD, 相当于真实情况下的 BootLoader

从 Flash 装入主存, 其中的 RW 段是紧随 RO 段存放, 并没有按 RWBase 指定位置存放。这个工作需要代码段自行完成。

2 Task1: 安装 ADS1.2

在 WIN11 操作系统上安装 ADS1.2 是很费功夫的一件事, 强烈建议使用虚拟机进行操作! 下面是在 WIN11 上安装 ADS1.2 的操作流程。首先, 下载安装包, 执行 Setup.exe 然后一路 Next。这是最基本的下载流程, 事实上鲜有人直接成功。在这里列出来一下我碰到的几个坑:

1. 不要在 Program Files(x86) 文件夹下去下载, 把安装包下下来之后, 在 Program Files 文件夹下创建 ARM 文件夹, 在 ARM 文件夹下再创建 ADSv1.2, 然后把文件下载到这个目录之下。这可能是因为 64 位/32 位之间的差别或者路径有括号的原因。

2. 如果碰到下载进度条到百分之百卡着不动，直接启动任务管理器退出任务，然后在你安装包文件夹的 crack 文件夹中找到 license.dat 文件，把这个文件粘贴到 ADSv1.2 的 license 文件夹下，就免去了最后选择注册表的那一步。

3. 如果碰到安装不成功需要卸载，会碰到点开 Setup.exe 一直是 modify/repair/remove 这几个选项的情况，即使删了也无济于事。网上对于这个 bug 有很多攻略，涉及到删注册表，这里提供一下在 WIN11 环境 (大概率 WIN10 也需要) 下要补充删除的注册表的几个地方：

HKEY_LOCAL_MACHINE\SOFTWARE\WOW6432Node\ARM Limited\ARM Developer Suite

HKEY_LOCAL_MACHINE\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall 下查找 {406FBBD8-EAFA-11D4-8FD0-0010B5688C67}

这样，大多数 ADSv1.2 安装的坑就被我们解决了，现在可以启动 CodeWarrior 来进行汇编了！

3 Task2: 建立工程并进行汇编

现在我们创建一个新 Project，在 DebugRel Settings 中配置环境。主要有以下几点需要修改：

ARM Assembler->Target->Architecture or Processor->ARM920T

ARM C Compiler->Target and Source->Architecture or Processor->ARM920T

ARM Linker->Output->Linktype->Simple

ARM Linker->Output->R0 Base->0x0

ARM Linker->Options->Image entry point->0x0

Target Setting->Post-linker->ARM fromELF

ARM FromELF -> Plain binary

配置完成之后，我们保存 PPT 中的源代码到 main.s 文件，然后在 project 里导入它，源代码如下：

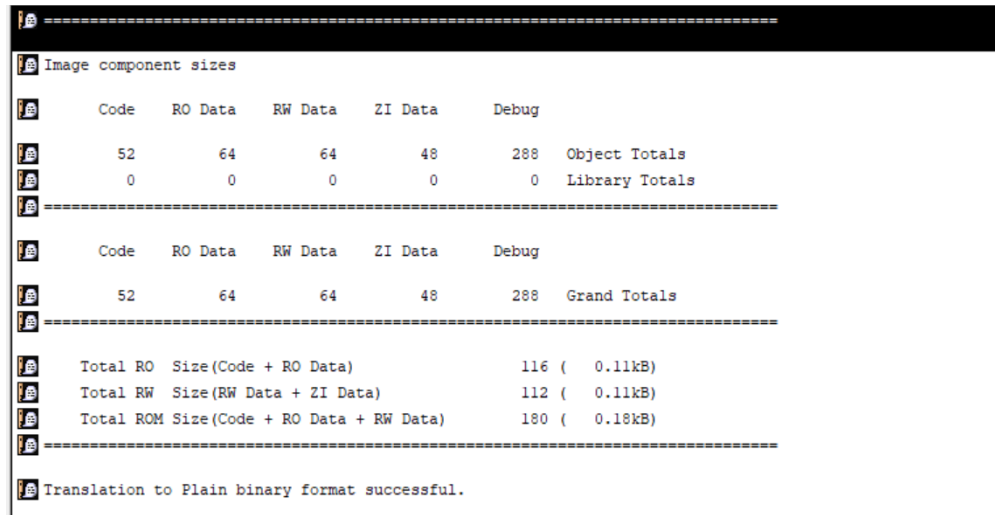
```
1
2  IMPORT  |Image$$RO$$Base|
3  IMPORT  |Image$$RO$$Limit|
4  IMPORT  |Image$$RW$$Base|
5  IMPORT  |Image$$RW$$Limit|
6  IMPORT  |Image$$ZI$$Base|
7  IMPORT  |Image$$ZI$$Limit|
8
9  AREA MyCode, CODE, READONLY
10 ENTRY
11 LDR     R0, =0x53002345
12 LDR     R1, =0x0
13 STR     R1, [R0]
14 LDR     R2, =Str1
15 LDR     R3, =Str2
16 LDR     R4, =Str3
17 LDR     R5, =DateSpace
18 B      .
19 AREA MyROData, DATA, READONLY, ALIGN=6
20 Str1
21 DCB     "This is my ROData."
22 AREA MyRWData0, DATA, READWRITE, ALIGN=5
23 Str2
24 DCB     "This is my RWData0."
```

```

25  AREA MyRWData1, DATA, READWRITE, ALIGN=4
26  Str3
27      DCB      "This is my RWData1."
28  AREA MyZIData, DATA, READWRITE, NOINIT, ALIGN=4
29  DateSpace
30      SPACE    40
31      END

```

汇编后的截图如下：



Code	RO Data	RW Data	ZI Data	Debug	
52	64	64	48	288	Object Totals
0	0	0	0	0	Library Totals
=====					
Code	RO Data	RW Data	ZI Data	Debug	
52	64	64	48	288	Grand Totals
=====					
Total RO	Size(Code + RO Data)			116 (	0.11kB)
Total RW	Size(RW Data + ZI Data)			112 (	0.11kB)
Total ROM	Size(Code + RO Data + RW Data)			180 (	0.18kB)
=====					
Translation to Plain binary format successful.					

图 1: 汇编完成

AXD Debugger 界面如下：

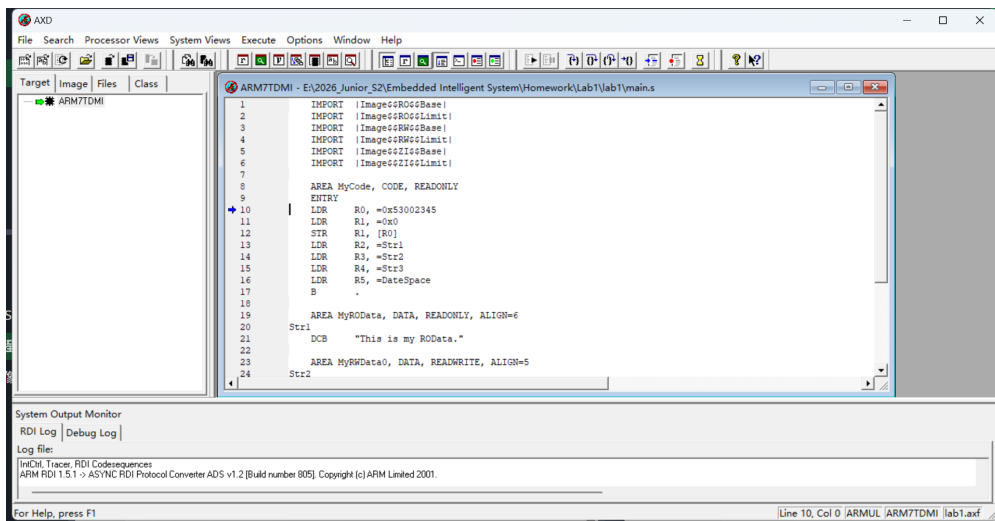


图 2: AXD Debugger

4 Task 3: 观察映像文件的组织

4.1 观察各段的安排次序

我们在 AXD Debugger 中的 Disassembly 中观察代码在内存里的真实排布，下面是部分结果截图：

```
Image$$RO[0xe59f0018]  ldr      r0,0x00000020 ; = #0x53002345
00000004 [0xe3a01000]  mov      r1,#0
00000008 [0xe5801000]  str      r1,[r0,#0]
0000000c [0xe59f2010]  ldr      r2,0x00000024 ; = #0x00000040
00000010 [0xe59f3010]  ldr      r3,0x00000028 ; = #0x00000080
00000014 [0xe59f4010]  ldr      r4,0x0000002c ; = #0x000000a0
00000018 [0xe59f5010]  ldr      r5,0x00000030 ; = #0x000000c0
0000001c [0xeaffffff]  b        0x1c ; (Image$$RO$$Base + 0x1c)
00000020 [0x53002345]  dcd      0x53002345 E$.S
00000024 [0x00000040]  dcd      0x00000040 @...
00000028 [0x00000080]  dcd      0x00000080 ....
0000002c [0x000000a0]  dcd      0x000000a0 ....
00000030 [0x000000c0]  dcd      0x000000c0 ....
00000034 [0x00000000]  dcd      0x00000000 ....
00000038 [0x00000000]  dcd      0x00000000 ....
0000003c [0x00000000]  dcd      0x00000000 ....
Str1      [0x73696854]  dcd      0x73696854 This
00000044 [0x20736920]  dcd      0x20736920 is
00000048 [0x5220796d]  dcd      0x5220796d my R
0000004c [0x7461444f]  dcd      0x7461444f ODat
```

图 3: Result 1

```
0000007c [0x00000000]  dcd      0x00000000 ....
Str2      [0x73696854]  dcd      0x73696854 This
00000084 [0x20736920]  dcd      0x20736920 is
00000088 [0x5220796d]  dcd      0x5220796d my R
0000008c [0x74614457]  dcd      0x74614457 WDat
00000090 [0x002e3061]  dcd      0x002e3061 a0..
00000094 [0x00000000]  dcd      0x00000000 ....
00000098 [0x00000000]  dcd      0x00000000 ....
0000009c [0x00000000]  dcd      0x00000000 ....
Str3      [0x73696854]  dcd      0x73696854 This
000000a4 [0x20736920]  dcd      0x20736920 is
000000a8 [0x5220796d]  dcd      0x5220796d my R
000000ac [0x74614457]  dcd      0x74614457 WDat
000000b0 [0x002e3161]  dcd      0x002e3161 al..
000000b4 [0x00000000]  dcd      0x00000000 ....
000000b8 [0x00000000]  dcd      0x00000000 ....
000000bc [0x00000000]  dcd      0x00000000 ....
Image$$ZI[0xe7ff0010]  dcd      0xe7ff0010 ....
000000c4 [0xe800e800]  dcd      0xe800e800 ....
000000c8 [0xe7ff0010]  dcd      0xe7ff0010 ....
```

图 4: Result 2

观察上面的结果，我们得到了各段的安排次序：

代码段 RO Code：是从 0x00000000 开始的。

RO Data (即 Str1)，对应的地址是 0x00000040。这正好对应了代码中写的 ALIGN=6，意思是地址必须是 $2^6 = 64$ 的倍数。为了凑够这个地址，在地址 0x34 到 0x3c 之间，编译器自动塞了一堆 0x00000000 来占位。

RW Data 0 (即 Str2)，对应的地址是 0x00000080。我们的程序写了 ALIGN=5，即 $2^5 = 32$ 的倍数，这验证了这一点。

RW Data 1 (即 Str3)：标号 Str3 的地址是 0x000000a0。我们写了 ALIGN=4，即 $2^4 = 16$ 的倍数，0xa0 也是 16 的倍数。ZI Data (DataSpace)：观察地址 00000018 的指令：ldr r5, 0x00000030 ; = #0x000000c0，这说明 DataSpace 的实际地址被分配在了 0x000000c0。而在截图中我们也能观察到这一点。

同时，我们观察到，各段在主存的存放地址以及 6 个链接器的符号常量指示的地址是相同的。因为没有提出特殊要求，链接器就顺其自然地把它们挨着放了。

4.2 修改 RO Base 后观察

接下来，我们把 ROBase 修改为 0x800，链接汇编，观察修改之后各段的安排次序，下面是部分截图：

```

000007fc [0xe800e800] stmda r0,[r11,r13-pc]
+ Image$$RO[0xe59f0018] ldr r0,0x00000820 ; = #0x53002345
00000804 [0xe3a01000] mov r1,#0
00000808 [0xe5801000] str r1,[r0,#0]
0000080c [0xe59f2010] ldr r2,0x00000824 ; = #0x00000840
00000810 [0xe59f3010] ldr r3,0x00000828 ; = #0x00000880
00000814 [0xe59f4010] ldr r4,0x0000082c ; = #0x000008a0
00000818 [0xe59f5010] ldr r5,0x00000830 ; = #0x000008c0
0000081c [0xeaffffff] b 0x81c ; (Image$$RO$$Base + 0x1c)
00000820 [0x53002345] dcd 0x53002345 E#.S
00000824 [0x00000840] dcd 0x00000840 @...
00000828 [0x00000880] dcd 0x00000880 ....
0000082c [0x000008a0] dcd 0x000008a0 ....
00000830 [0x000008c0] dcd 0x000008c0 ....
00000834 [0x00000000] dcd 0x00000000 ....
00000838 [0x00000000] dcd 0x00000000 ....
0000083c [0x00000000] dcd 0x00000000 ....
Str1 [0x73696854] dcd 0x73696854 This
00000844 [0x20736920] dcd 0x20736920 is
00000848 [0x5220796d] dcd 0x5220796d my R

```

图 5: Result 1

```

0000087c [0x00000000] dcd 0x00000000 ....
Str2 [0x73696854] dcd 0x73696854 This
00000884 [0x20736920] dcd 0x20736920 is
00000888 [0x5220796d] dcd 0x5220796d my R
0000088c [0x74614457] dcd 0x74614457 WDat
00000890 [0x002e3061] dcd 0x002e3061 a0..
00000894 [0x00000000] dcd 0x00000000 ....
00000898 [0x00000000] dcd 0x00000000 ....
0000089c [0x00000000] dcd 0x00000000 ....
Str3 [0x73696854] dcd 0x73696854 This
000008a4 [0x20736920] dcd 0x20736920 is
000008a8 [0x5220796d] dcd 0x5220796d my R
000008ac [0x74614457] dcd 0x74614457 WDat
000008b0 [0x002e3161] dcd 0x002e3161 al..
000008b4 [0x00000000] dcd 0x00000000 ....
000008b8 [0x00000000] dcd 0x00000000 ....
000008bc [0x00000000] dcd 0x00000000 ....
Image$$ZI[0xe7ff0010] dcd 0xe7ff0010 ....
000008c4 [0xe800e800] dcd 0xe800e800 ....
000008c8 [0xe7ff0010] dcd 0xe7ff0010 ....

```

图 6: Result 2

我们观察各段的安排顺序：代码段 RO Code：变为从 0x00000800 开始。

Str1 (RO Data) 现在位于 0x00000840。

Str2 (RW Data 0) 现在位于 0x00000880。

Str3 (RW Data 1) 现在位于 0x000008a0。

我们也观察到，各段在主存的存放地址以及 6 个链接器的符号常量指示的地址是相同的。因为 RWBase 我们还没有填写数字，所以没有产生任何错位。

4.3 修改 RW Base 后观察

接下来，我们把 RWBase 修改为 0xC00，链接汇编，再次观察，下面是部分截图：

```

+ Image$$RO[0xe59f0018] ldr r0,0x00000820 ; = #0x53002345
00000804 [0xe3a01000] mov r1,#0
00000808 [0xe5801000] str r1,[r0,#0]
0000080c [0xe59f2010] ldr r2,0x00000824 ; = #0x00000840
00000810 [0xe59f3010] ldr r3,0x00000828 ; = #0x00000c00
00000814 [0xe59f4010] ldr r4,0x0000082c ; = #0x00000c20
00000818 [0xe59f5010] ldr r5,0x00000830 ; = #0x00000c40
0000081c [0xeaffffff] b 0x81c ; (Image$$RO$$Base + 0x1c)
00000820 [0x53002345] dcd 0x53002345 E#.S
00000824 [0x00000840] dcd 0x00000840 @...
00000828 [0x00000c00] dcd 0x00000c00 ....
0000082c [0x00000c20] dcd 0x00000c20 ...
00000830 [0x00000c40] dcd 0x00000c40 @...
00000834 [0x00000000] dcd 0x00000000 ....
00000838 [0x00000000] dcd 0x00000000 ....
0000083c [0x00000000] dcd 0x00000000 ....
Str1 [0x73696854] dcd 0x73696854 This
00000844 [0x20736920] dcd 0x20736920 is

```

图 7: Result 1

```

00000870 [0x00000000] dcd 0x00000000 ....
00000874 [0x00000000] dcd 0x00000000 ....
00000878 [0x00000000] dcd 0x00000000 ....
0000087c [0x00000000] dcd 0x00000000 ....
Image$$RO[0x73696854] dcd 0x73696854 This
00000884 [0x20736920] dcd 0x20736920 is
00000888 [0x5220796d] dcd 0x5220796d my R
0000088c [0x74614457] dcd 0x74614457 WDat
00000890 [0x002e3061] dcd 0x002e3061 a0..
00000894 [0x00000000] dcd 0x00000000 ....
00000898 [0x00000000] dcd 0x00000000 ....
0000089c [0x00000000] dcd 0x00000000 ....
000008a0 [0x73696854] dcd 0x73696854 This
000008a4 [0x20736920] dcd 0x20736920 is
000008a8 [0x5220796d] dcd 0x5220796d my R
000008ac [0x74614457] dcd 0x74614457 WDat
000008b0 [0x002e3161] dcd 0x002e3161 al..
000008b4 [0x00000000] dcd 0x00000000 ....
000008b8 [0x00000000] dcd 0x00000000 ....

```

图 8: Result 2

这次的现象和之前有着不小的差距：

在链接器规划的运行地址（执行视图）层面，各段次序出现跳跃，依次为：代码段 → RO 数据段 → [地址断层] → RW 数据段 → ZI 数据段。

具体的运行地址变为：

代码段 (RO Code) 起始地址保持为: 0x00000800。

Str1 (RO Data) 保持位于: 0x00000840。

Str2 (RW Data 0) 运行地址跳跃至: 0x00000C00。

Str3 (RW Data 1) 运行地址变为: 0x00000C20。

DateSpace (ZI Data) 运行地址变为: 0x00000C40。

但是, 在主存的真实存放地址 (加载视图) 中观察发现: RW 数据段 (Str2、Str3) 依然紧跟在 RO 数据段之后, 实际停留在 0x00000880 和 0x000008A0 的位置, 并没有真正移动到 0xC00 所在的内存区域。

4.4 观察主存的存放地址以及符号常量指示的地址

在上面的流程之中, 我们已经完成了观察主存的存放地址以及符号常量指示的地址的任务。下面我们总结一下:

在把 ROBase 和 RWBase 都设置之后, 我们观察到主存的真实存放地址和链接器指定的符号地址不一样。我们在设置里把 RW 段的运行地址指到了 0xC00, 但实际去内存里看, Str2 这串数据根本没搬过去, 依然在代码段后面。这是因为在开发板上, 程序是烧录在 Flash 里的。为了省空间, RW 默认挨着代码 RO 存放, 这就是存放地址。但程序跑起来后, 变量是要被修改的, Flash 不能写, 所以必须在 RAM 里给它划一块新地盘 (比如 0xC00), 这就是运行地址。

要把数据从 Flash 搬运到 RAM, 需要 Bootloader 来干这个活。但我们的纯汇编代码里没有包含这段代码, 数据自然就只能在存放地址中, 去不了链接器规划的地方了。

4.5 调整源代码中各段顺序, 再观察次序、地址

接下来, 我们调整源代码中各段顺序, 汇编后再观察各段的次序、地址。这一步实际上就是在验证: 是代码里写的先后顺序决定了内存排布, 还是编译器有其他的排布规则。于是我们还原最开始的链接环境, 故意打乱代码顺序, 打乱顺序后的代码如下:

```
1  IMPORT |Image$$RO$$Base|
2  IMPORT |Image$$RO$$Limit|
3  IMPORT |Image$$RW$$Base|
4  IMPORT |Image$$RW$$Limit|
5  IMPORT |Image$$ZI$$Base|
6  IMPORT |Image$$ZI$$Limit|
7
8  AREA MyRWData0, DATA, READWRITE, ALIGN=5
9  Str2
10  DCB      "This is my RWData0."
11
12  AREA MyCode, CODE, READONLY
13  ENTRY
14  LDR      R0, =0x53002345
15  LDR      R1, =0x0
16  STR      R1, [R0]
17  LDR      R2, =Str1
18  LDR      R3, =Str2
19  LDR      R4, =Str3
20  LDR      R5, =DateSpace
21  B        .
22
```



```

23 AREA MyROData, DATA, READONLY, ALIGN=6
24 Str1
25 DCB "This is my ROData."
26
27 AREA MyRWData1, DATA, READWRITE, ALIGN=4
28 Str3
29 DCB "This is my RWData1."
30
31 AREA MyZIData, DATA, READWRITE, NOINIT, ALIGN=4
32 DateSpace
33 SPACE 40
34
35 END

```

我们把 RW 段挪到了最前面，甚至在代码段之前，之后我们链接编译，观察各段的次序，地址。结果如下：

```

➤ Image$$RC [0x59f0018] ldr r0,0x00000020 ; = #0x53002345
00000004 [0xe3a01000] mov r1,#0
00000008 [0xe5801000] str r1,[r0,#0]
0000000c [0xe59f2010] ldr r2,0x00000024 ; = #0x00000040
00000010 [0xe59f3010] ldr r3,0x00000028 ; = #0x00000080
00000014 [0xe59f4010] ldr r4,0x0000002c ; = #0x000000a0
00000018 [0xe59f5010] ldr r5,0x00000030 ; = #0x000000c0
0000001c [0xeaffffff] b 0x1c ; (Image$$RO$$Base + 0x1c)
00000020 [0x53002345] dcd 0x53002345 E$.S
00000024 [0x00000040] dcd 0x00000040 @...
00000028 [0x00000080] dcd 0x00000080 ....
0000002c [0x000000a0] dcd 0x000000a0 ....
00000030 [0x000000c0] dcd 0x000000c0 ....
00000034 [0x00000000] dcd 0x00000000 ....
00000038 [0x00000000] dcd 0x00000000 ....
0000003c [0x00000000] dcd 0x00000000 ....
Str1 [0x73696854] dcd 0x73696854 This
00000044 [0x20736920] dcd 0x20736920 is
00000048 [0x5220796d] dcd 0x5220796d my R

```

图 9: Result 1

```

00000074 [0x00000000] dcd 0x00000000 ....
00000078 [0x00000000] dcd 0x00000000 ....
0000007c [0x00000000] dcd 0x00000000 ....
Str2 [0x73696854] dcd 0x73696854 This
00000084 [0x20736920] dcd 0x20736920 is
00000088 [0x5220796d] dcd 0x5220796d my R
0000008c [0x74614457] dcd 0x74614457 WDat
00000090 [0x002e3061] dcd 0x002e3061 a0..
00000094 [0x00000000] dcd 0x00000000 ....
00000098 [0x00000000] dcd 0x00000000 ....
0000009c [0x00000000] dcd 0x00000000 ....
Str3 [0x73696854] dcd 0x73696854 This
000000a4 [0x20736920] dcd 0x20736920 is
000000a8 [0x5220796d] dcd 0x5220796d my R
000000ac [0x74614457] dcd 0x74614457 WDat
000000b0 [0x002e3161] dcd 0x002e3161 al..
000000b4 [0x00000000] dcd 0x00000000 ....
000000b8 [0x00000000] dcd 0x00000000 ....

```

图 10: Result 2

我们观察到，即使在源代码里把 Str2 提到最前面，结果依然没有任何变化。这是因为 ARM 链接器在生成映像文件时，遵循其固定的段放置规则，而不是简单地按照源代码中的书写顺序排列。链接器会自动根据段的属性进行分类和排序，默认是 CODE>ROData>RWData>ZIData。

4.6 改变各段对齐方式与各数据段长度，再进行观察

首先，我们改变对齐方式，把 MyROData 的 ALIGN=6 改成 ALIGN=2，把 MyRWData0 的 ALIGN=5 改成 ALIGN=8。

其次，再改变各数据段长度，把 str1 改为 “Short”，把 str2 改为 “IT IS A VERY LONG STRING”，具体代码如下：

```

1 AREA MyROData, DATA, READONLY, ALIGN=2
2 Str1
3 DCB "Short"
4
5 AREA MyRWData0, DATA, READWRITE, ALIGN=8
6 Str2
7 DCB "IT IS A VERY LONG STRING."

```

之后我们观察运行后的结果，截图如下：

Str1	[0x726f6853]	dcd	0x726f6853	Shor
00000038	[0x00000074]	dcd	0x00000074	t...
Image\$\$RW	[0x00000000]	dcd	0x00000000	
00000040	[0x00000000]	dcd	0x00000000	
00000044	[0x00000000]	dcd	0x00000000	
00000048	[0x00000000]	dcd	0x00000000	
0000004c	[0x00000000]	dcd	0x00000000	
00000050	[0x00000000]	dcd	0x00000000	
00000054	[0x00000000]	dcd	0x00000000	
00000058	[0x00000000]	dcd	0x00000000	
0000005c	[0x00000000]	dcd	0x00000000	
00000060	[0x00000000]	dcd	0x00000000	
00000064	[0x00000000]	dcd	0x00000000	
00000068	[0x00000000]	dcd	0x00000000	
0000006c	[0x00000000]	dcd	0x00000000	
00000070	[0x00000000]	dcd	0x00000000	
00000074	[0x00000000]	dcd	0x00000000	
00000078	[0x00000000]	dcd	0x00000000	
0000007c	[0x00000000]	dcd	0x00000000	
00000080	[0x00000000]	dcd	0x00000000	
00000084	[0x00000000]	dcd	0x00000000	

图 11: Result 1

000000e4	[0x00000000]	dcd	0x00000000	
000000e8	[0x00000000]	dcd	0x00000000	
000000ec	[0x00000000]	dcd	0x00000000	
000000f0	[0x00000000]	dcd	0x00000000	
000000f4	[0x00000000]	dcd	0x00000000	
000000f8	[0x00000000]	dcd	0x00000000	
000000fc	[0x00000000]	dcd	0x00000000	
Str2	[0x49205449]	dcd	0x49205449	IT I
00000104	[0x20412053]	dcd	0x20412053	S A
00000108	[0x59524556]	dcd	0x59524556	VERY
0000010c	[0x4e4f4c20]	dcd	0x4e4f4c20	LON
00000110	[0x54532047]	dcd	0x54532047	G ST
00000114	[0x474e4952]	dcd	0x474e4952	RING
00000118	[0x0000002e]	dcd	0x0000002e	
0000011c	[0x00000000]	dcd	0x00000000	
00000120	[0x00000000]	dcd	0x00000000	
00000124	[0x00000000]	dcd	0x00000000	
00000128	[0x00000000]	dcd	0x00000000	
0000012c	[0x00000000]	dcd	0x00000000	
00000130	[0x00000000]	dcd	0x00000000	
00000134	[0x00000000]	dcd	0x00000000	
00000138	[0x00000000]	dcd	0x00000000	

图 12: Result 2

我们发现，由于将 MyRWData0 段的对齐方式设为了 ALIGN=8，观察 AXD 内存发现，前一个段 Str1 结束于 0x3C，但 Str2 的起始地址并没有顺延，而是强制对齐到了 0x00000100。中间产生了从 0x00000040 到 0x000000FC 共 192 字节的空隙。所以，对齐数值 n 越大，地址分配的“颗粒度”就越粗。在这样的情况下，过大的对齐设置会导致内存利用率低下，即会产生大量的内存碎片。

5 Task 4: 观察指定的伪指令 LDR R0,=0x53002345

5.1 观察文字池

首先我们找找文字池在哪里，观察下面的截图：

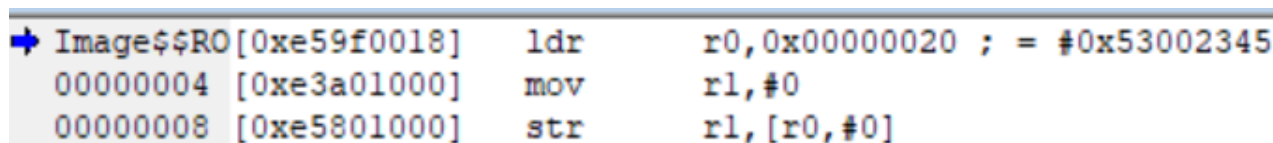
00000014	[0xe59f4010]	ldr	r4,0x0000002c ; = #0x000000a0
00000018	[0xe59f5010]	ldr	r5,0x00000030 ; = #0x000000c0
0000001c	[0xeaffffffe]	b	0x1c ; (Image\$\$R0\$\$Base + 0x1c)
00000020	[0x53002345]	dcd	0x53002345 E#.S
00000024	[0x00000040]	dcd	0x00000040 @...
00000028	[0x00000080]	dcd	0x00000080

图 13: 文字池

在跳转指令 B 之后的就是文字池。通常链接器会在代码段的末尾，或者在无法跨越的跳转指令之后，开辟一块空间存放一些“放不下”的常数。比如 LDR 伪指令中的 0x53002345，因为这个数太大，无法作为立即数编码进指令，所以编译器把它放到了这里。我们观察地址 0x00000020 里存储的内容，0x53002345，其就是文字池中存储的伪指令中的内容。

5.2 观察其对应的机器指令，说明对应的汇编指令及寻址方式

根据上课学习的 LDR 伪指令的原理，因为一条指令本身占 32 位，它就不可能在指令内部再塞下一个 32 位的常数。0x53002345 不能通过八位数循环右移偶数次得到，所以他不能转换为一个 MOV 指令，而是把这个大数存放在代码段后面的文字池，然后把原来的伪指令替换成一条真正的内存加载指令。我们截图验证一下：



```
➔ Image$$RO[0xe59f0018]    ldr    r0,0x00000020 ; = #0x53002345
00000004 [0xe3a01000]    mov    r1,#0
00000008 [0xe5801000]    str    r1,[r0,#0]
```

图 14: LDR 伪指令

上图的第一条指令，其实就是在寻找的 LDR 伪指令汇编后的指令。其机器码为 0xe59f0018。这就是 CPU 真正执行的 32 位机器指令。ldr r0, 0x00000020 是 AXD 算好后的结果。它发现最后是要去地址 0x20 拿数据，所以直接把目标地址写出来了。

我们观察到一点，机器指令的最后两位是 18，这一点是为什么呢？实际上，根据 ARM 的规定，PC 指针总是指向“当前指令 + 8 字节”的位置（流水线工作的原因），当前指令地址为 0x00000000，即 PC 是 0x00000008，偏移量为 18，PC+ 偏移量就是 0x20 了。这也就说明了，ldr r0, 0x00000020 让我们看到。的背后逻辑就是 LDR R0, [PC, 0x18]。

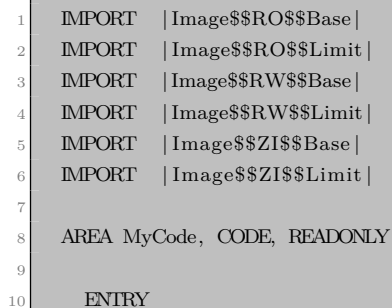
我们再来详细说一遍这条伪指令的工作流程。首先，因为立即数是 0x53002345，其无法通过八位数循环右移偶数次得到，所以其被存入文字池，然后汇编器计算出偏移量，之后，伪指令直接被汇编器机器指令，其机器码为 0xe59f0018(为什么偏移量是 18 上面有写)，之后反汇编为 ldr r0, 0x00000020 让我们看到。

作为对比，我们看看第二条指令 LDR R1, =0x0 反汇编之后的结果。在上面的图 14 之中，我们能看到其最后是 MOV r1, 0。这也补足了上面的另一种情况：立即数可以通过八位数循环右移偶数次得到。这种情况下，汇编器直接将 LDR 伪指令汇编成一个 MOV 指令，其好处就是不用把立即数存储到主存之中，这便提高了程序的读写效率。

6 Task5：增加其他指令，观察执行情况

6.1 增加堆栈指令，转移指令

我们编写新程序，增加堆栈指令，转移指令。代码如下：



```
1  IMPORT |Image$$RO$$Base|
2  IMPORT |Image$$RO$$Limit|
3  IMPORT |Image$$RW$$Base|
4  IMPORT |Image$$RW$$Limit|
5  IMPORT |Image$$ZI$$Base|
6  IMPORT |Image$$ZI$$Limit|
7
8  AREA MyCode, CODE, READONLY
9
10  ENTRY
11
```

```

12  MOV     SP, #0x400
13  LDR     R0, =0x11111111
14  LDR     R1, =0x22222222
15  STMFD   SP!, {R0, R1}
16  MOV     R2, #0x0
17  CMP     R0, R1
18  BNE     SkipNext
19  MOV     R2, #0xFF
20
21  SkipNext
22
23  LDMFD   SP!, {R3, R4}
24  B       .
25  END

```

这份代码初始化了一个堆栈，然后把两个大常数放进寄存器 R1、R2，之后把寄存器中的两个值压入堆栈，判断这两个数是否相等，如果不相等就把这两个数从堆栈中取出存到 R3、R4 中。

下面我们分别对压入堆栈的 STMFD 指令，跳转 BNE SkipNext 指令来进行机器码的分析：

STMFD SP!, {R0, R1}：其机器码为 0xe92d0003，转换为二进制为 1110 | 100 | 1 | 0 | 0 | 1 | 0 | 1101 | 0000000000000011。我们一个一个部分分析，前四位 1110 是条件码，含义是 AL(Always)，代表这条指令无条件执行。27-20 位是指令类型与标志位，这一部分定义了这是一条多寄存器存储指令，并规定了它怎么存。Bit24 为 1 表示在存数据前先改变地址；Bit23 为 0 表示地址递减；Bit 21 为 1 表示存完之后，把新的地址写回 SP；Bit20 表示这是存储。Bits 19-16 为 1101，即十进制的 13，表示 SP，这告诉 CPU 以 SP 指向的地址作为基准来操作；最后 Bit15-0 是寄存器列表位图，后两位是 1 表示选中了 R0 和 R1。

BNE SkipNext：其机器码为 0x1a000000，其条件码为 1，表示 NE (Not Equal) 条件，只有当 CPSR 寄存器里的 Z 标志位为 0 时，这条指令才会触发；条件码后面的四位是 1010，其前面的 101 表示跳转指令；最后的 0 表示是普通跳转。

6.2 单步执行，观察寄存器情况

我们来分析单步执行的情况：

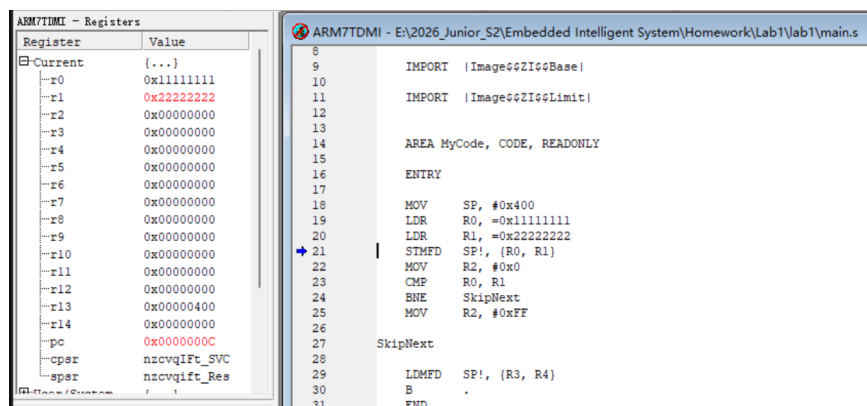


图 15: Step 1

上图是执行三个指令后寄存器的情况，我们看到 R1, R2 中已经有了数据，而且 R13 中存储到了我们初始化的堆栈指针。

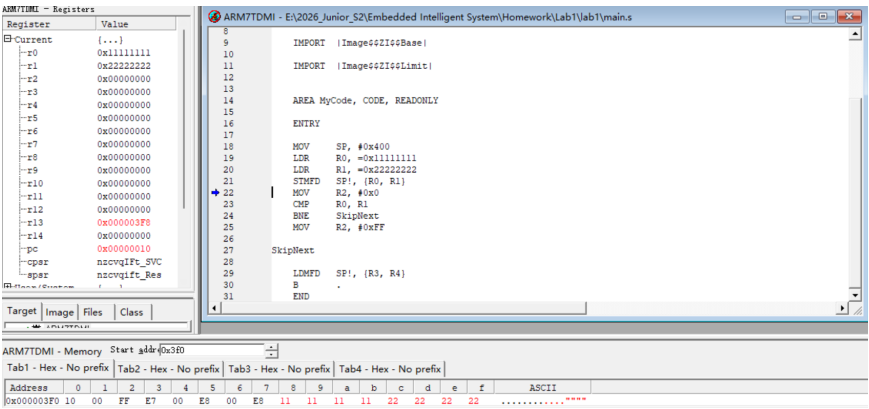


图 16: Step 2

上图是再执行一句的情况，我们发现堆栈指针现在指向 0x000003F8，这个由于我们是满递减堆栈，所以数据应该存储在 0x000003F8-0x00000400 之中。我们看看底下的内存区域，发现 0x000003F8-0x000003FB 存储的是 R0, 0x000003FC-0x000003FF 存储的是 R1，这也验证了课上学到的多寄存器寻址的时候，寄存器列表中编号小的寄存器存低地址的规则。

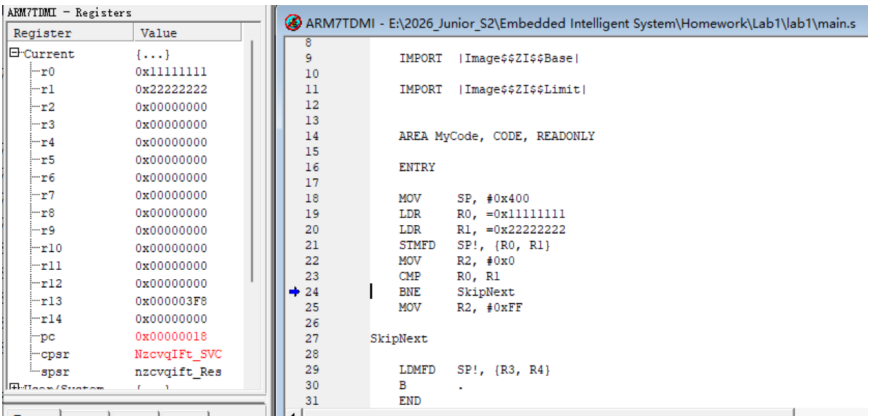


图 17: Step 3

上图是再执行两句之后的情况，我们发现 cpsr 寄存器在执行过 CMP 语句之后发生了改变，因为比较结果不同。

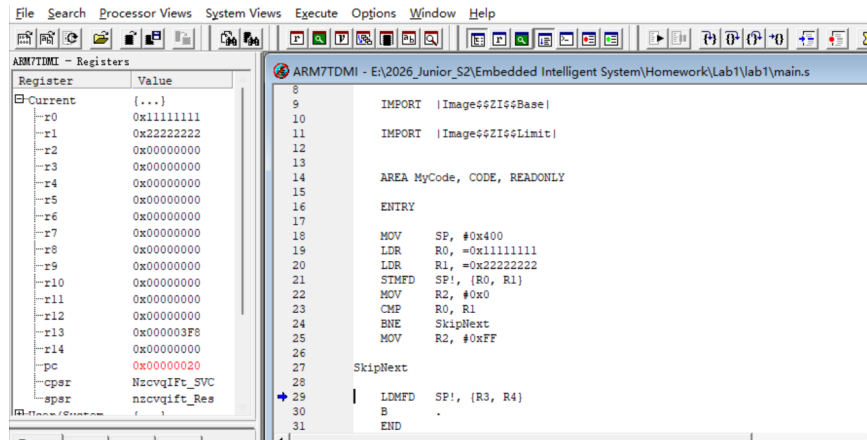


图 18: Step 4

再执行一步之后，我们执行到了跳转指令。由于 BNE 是如果不同则跳转，之前执行 CMP 语句的时候其比较结果是不同的，所以发生了跳转。我们看到蓝色箭头跳转到了 LDMFD 这条语句的地方。

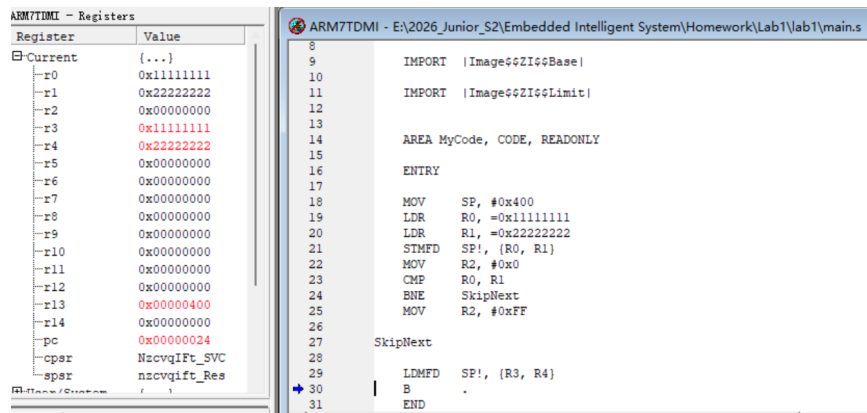


图 19: Step 5

再执行一步，R3 和 R4 已经被堆栈中的两个数据所填充。之后我们继续执行，就进入了死循环，这样可以防止程序把后面的数据当作代码处理。

6.3 对各段地址解释的理解

在嵌入式系统启动时，AXD 加载镜像模拟了 Flash 加载视图，为了存储连续性，RW 段（变量初始值）物理上紧随 RO 段（代码）存放。然而，程序执行时要求变量位于可读写的 RAM 中（即 RWBase 指定的执行视图）。由于硬件和调试器不会自动处理存放地址与运行地址的不一致，因此必须由 Bootloader 在进入主程序前，手动将 RW 数据从其存放位置搬移到目标运行地址，并对 ZI 段区域清零。只有完成这种搬运，程序才能在预定的内存空间正确访问并修改变量。而这个任务就是我们程序员所应该完成的。